Shadowseed: Remembering Without Trusting

A Validation-Gated Memory Architecture for Language Model Systems

H. Visser, ChatGPT, Claude, Kimi, and GLM
E-AI-MODEL

August 2026

Abstract

Memory is easy to add to a language-model system. Deciding when a remembered item deserves influence is harder. Existing systems retrieve documents, retain conversational facts, store reflections, or adapt retrieval to the current query. A narrower case remains awkward: the system suspects that something is missing from an answer or context without any warrant to treat that suspicion as true. Shadow Seed Learning (SSL) addresses that case by storing candidate omissions as explicitly untrusted memory. A seed can remain present with no steering authority: its trace may be positive at weight zero. Authority changes pass through a typed Validation Gate. Verified external support carries provenance; repeated evidence is idempotent at a defined source-and-signal boundary; unresolved contradictions block use; and a second, current-version authorization check is required when a seed is about to affect retrieval or generation. The implementation separates two conversation modes. The product-oriented live path performs one generation, stores the visible answer, and requires explicit verified evidence before recurrence can gain authority. The evaluation path retains an isolated baseline for controlled comparison. Both allowed and denied influence attempts are recorded. We describe the state model, lifecycle, evidence rules, contradiction handling, point-of-use contract, and prompt-data boundary. Evaluation of the reference implementation focuses on assurance of these contracts. Answer quality is outside the present evaluation. The repository gates its tests with linting, branch coverage, multi-version CI, build, clean-install, Workbench, portability, and release checks. These results establish implementation contracts; general answer improvement, semantic correctness of proposed omissions, universal security, and production readiness remain outside their evidential scope. SSL is therefore presented as a research-ready architecture for retaining uncertain candidate context without granting it automatic authority.

1 Introduction

Fluent language-model answers can omit a dependency, a qualification, a causal boundary, a stakeholder, or a question that would change their interpretation. Reliable detection of such absences remains unresolved. AbsenceBench shows a marked gap between retrieving information that is present and identifying information that has been deliberately removed from a context [1]. QuestBench reaches the same problem from another direction: even strong models can struggle to identify the missing variable that must be requested before an underspecified reasoning task becomes solvable [2]. Long-term interaction adds another difficulty. LongMemEval finds substantial degradation when assistants must retrieve, update, and reason over information distributed across sessions [3].

A tempting response is to save every plausible omission and put it back into later prompts. That solves persistence by creating a different problem. A model-generated suspicion has crossed an epistemic boundary merely because it was written to memory. Repetition can then make a weak

hypothesis more available, retrieval can amplify it, and a later model may read the stored text with little indication of how uncertain it was when created.

SSL starts from a stricter premise: *remembered* and *authorized to influence* are different states. A detector may propose a candidate omission. The system may keep that candidate, recognize it later, compare it with external material, or record that it recurs. Keeping, recognizing, comparing, or counting a candidate leaves steering power unchanged. In the reference implementation, a newly created seed has positive trace and exactly zero weight. A Validation Gate controls authority changes. A point-of-use contract checks the current authorization again when the seed is about to affect retrieval, a probe, a warning, an answer, or another downstream action [4].

This separation changes the role of memory. Memory becomes a place where uncertain hypotheses can remain available for testing without carrying an endorsement. Several failure cases then become visible in the data model. A candidate may be present with little authority. Recurrence may support it when external evidence is absent. An earlier promotion may have gone stale. The candidate may be contradicted, expired, or simply irrelevant to the current turn.

The contribution of this paper is the architecture and its executable authority contracts. It formalizes candidate-omission memory as a stateful process with separate persistence and influence variables; routes new authority through typed, policy-bound Gate decisions; attaches provenance and identity to external support; treats contradiction as an explicit blocking record; and requires a current authorization at the moment of use. The accompanying software artifact implements these rules in a testable Python runtime and exposes the decision trail for inspection [4]. We evaluate whether those contracts are enforced. Open-ended answer quality lies outside the claims supported by the present artifact. That empirical question requires a separate evaluation with fixed tasks, blinded comparison, independent review, and a frozen implementation.

2 Problem setting

2.1 Candidate omissions are hypotheses

Consider an answer that reports an association and then moves directly to a recommendation. A detector may propose the candidate omission: "The answer does not establish whether the association is causal." The proposal can be useful even when it is wrong. It points to a specific absence that can be checked. Treating the sentence as a fact would collapse detection and validation into one operation.

SSL calls such a proposal a *seed*. A seed is intended to express one testable candidate absence. Atomicity is a design target with no semantic guarantee. Model-generated candidates can still be vague, compound, redundant, or based on a poor reading of the source. The architecture therefore grants no authority on the strength of a convincing formulation alone.

This choice differs from conventional memory tasks. Long-term conversational memory often asks whether a stored fact can be retrieved after many interactions [3]. Retrieval-augmented generation asks which external documents should be brought into context [5, 6]. Reflective agents store linguistic feedback that can guide later attempts [7]. In SSL, the stored object is uncertain by construction. The central problem is the controlled transition from a remembered hypothesis to bounded permission to influence.

2.2 Failure modes of unqualified memory

Four failure modes motivate the design.

Persistence can masquerade as confidence. A statement that reappears across turns may become salient simply because the detector repeats the same mistake. Recurrence records persistence. Correctness requires separate support.

Retrieved material can be wrong or hostile. RAG systems gain access to external knowledge at the cost of another trust boundary. PoisonedRAG demonstrates that a small number of maliciously inserted texts can steer answers in a retrieval system [8]. The security problem is broader for agents: AgentDojo shows how untrusted data returned by tools can contain instructions that interfere with the user's task [9]. A memory architecture should therefore preserve source identity and distinguish data from instructions.

Past authorization can become stale. A seed that was once supported can later receive contradictory evidence, lose authority, or change state. Caching an earlier "allowed" result would bypass the new state unless authorization is versioned.

A gate at storage time is insufficient. A promoted seed may be irrelevant to the current query. It may also become unsafe to use because a contradiction opened after promotion. Permission to remain in memory and permission to affect this action are separate decisions.

These cases suggest a design requirement that is stronger than "attach a confidence score." The system needs a traceable authority path whose inputs and state transitions can be inspected after the fact.

3 Related work

3.1 Retrieval and external memory

REALM and RAG established influential patterns for combining language models with non-parametric stores [5, 6]. Their main concern is access to external knowledge beyond the model's parameters. Later work makes retrieval conditional. Adaptive-RAG selects among no-retrieval, single-step retrieval, and iterative retrieval according to estimated query complexity [10]. Self-RAG trains a model to retrieve and critique its own generations through special reflection signals [11].

SSL complements these methods. A seed may eventually trigger retrieval, and retrieved evidence may support or oppose it. The remaining question concerns the authority of the candidate that initiated the search after the result returns. A retrieved document enters validation as evidence input; its presence alone leaves the seed unconfirmed. Source conflict and poisoned stores make that distinction consequential [8].

3.2 Long-term and reflective agent memory

MemGPT treats the context window as a constrained memory tier and moves information between fast and slow stores [12]. Reflexion retains verbal feedback in episodic memory so later trials can use lessons from earlier failures [7]. These systems demonstrate that persistent text can change subsequent model behavior without updating model parameters.

SSL focuses on a different object. A seed is born as a hypothesis about an omission, so retention contributes no evidence for its content. The architecture therefore gives the stored item two independent quantities: trace for presence and recurrence, and weight for steering authority. This is the main conceptual departure from memory mechanisms in which successful retrieval is already close to use.

3.3 Missing information and verification

AbsenceBench directly tests detection of omitted content and reports that strong language models remain substantially worse at finding absences than at retrieving inserted information [1]. QuestBench evaluates the ability to ask for a missing variable in underspecified reasoning problems and finds pronounced weaknesses on logic and planning tasks [2]. These benchmarks leave persistence unspecified. Together they motivate treating missing information as an explicit computational object.

Verification work addresses a related trust problem after generation. Chain-of-Verification drafts an answer, generates verification questions, answers them separately, and produces a revised response [13]. SSL moves part of that discipline into persistent state: a candidate can survive between turns without being promoted, and its future use remains conditional on logged authority. The two approaches are compatible. A verification result can be represented as a validation signal, provided its provenance and trust status are explicit.

3.4 Uncertainty and authority

Semantic entropy measures uncertainty over meanings in sampled model outputs and can identify a class of confabulations [14]. Such uncertainty estimates answer a different question from SSL authority. A low-entropy candidate can be unsupported. A high-entropy candidate can still merit investigation. SSL keeps model confidence separate from steering permission. Authority is a system-level decision based on typed signals and policy.

4 Shadow Seed Learning

4.1 State representation

Let a seed at time t be

$$s_t = (x, \tau_t, w_t, z_t, E_t, C_t, v_t), \tag{1}$$

where x is the candidate text, $\tau_t \geq 0$ is trace, $w_t \in [0, 1]$ is steering weight, z_t is lifecycle/authority status, E_t is the set of accepted evidence identities, C_t is the set of contradiction records, and $v_t \in \mathbb{N}_0$ is an authority version.

Creation obeys two invariants:

$$\tau_{t_0} > 0, \qquad w_{t_0} = 0. \tag{2}$$

Presence therefore carries no implication of influence:

$$\tau_t > 0 \;\not\Rightarrow\; w_t > 0. \tag{3}$$

The reference implementation uses the statuses `NEW`, `ACTIVE`, `DECAYING`, `DORMANT`, `PROMOTED`, and `EXPIRED`. Status mixes lifecycle and one authority boundary because `PROMOTED` is the state from which a seed may become eligible for influence. Eligibility remains derived from several fields; no free-standing mutable flag stores it.

4.2 Trace, decay, and reactivation

Trace records that a candidate exists in shadow memory and captures recurrence or reactivation. A time-to-live process reduces trace during periods without reinforcement. A dormant seed can

return through trigger or semantic matching, a transition referred to as Triggered Recall Time-to-Live (TrTL) in the implementation. Expiry is terminal. An expired seed has no weight, and the lifecycle prevents automatic resurrection.

The lifecycle deliberately permits a long-lived candidate with zero authority. A recurring suspicion may deserve continued observation without being allowed to shape the user's answer. A seed that once had authority can lose it through contradiction or expiry. Its historical record remains available for audit. Trace decay uses true half-life semantics, $\tau_{t+h} = \tau_t/2$. The default $h = 3\ln 2$ preserves the earlier calibrated curve $\exp(-t/3)$ while explicit configurations now mean what the parameter name states.

4.3 Typed validation signals

Validation uses typed signals. A generic confidence increment would erase distinctions among recurrence, external support, contradiction, and other evidence channels. A signal has a kind, direction, bounded strength, optional source reference, verification flag, independence flag, and reason. Implemented kinds include recurrence, trusted-source evidence, human feedback, retrieval, dialectic checks, probes, task outcomes, contradiction, and contradiction resolution [4].

The distinction between recurrence and external evidence is fixed in the type system. Recurrence means that a candidate, or a semantically similar candidate, appeared again. Crossing a recurrence threshold leaves the signal classified as recurrence. In the reference implementation, trusted external evidence is restricted to designated signal kinds and must carry a non-empty `source_ref` before it can count as verified support for a non-expired seed.

Evidence has an identity boundary. For an external support signal e, define

$$\kappa(e) = (\text{source\_ref}(e), \text{kind}(e)). \tag{4}$$

Replaying the same $\kappa(e)$ is idempotent and leaves authority unchanged. Independent confirmations of the same signal kind need distinct source references. The same source reference used under a different typed signal kind is represented as a different observation. The rule defines accounting identity while source independence remains a separate empirical question.

4.4 Validation Gate

A Gate policy receives the current read-only authority snapshot and a set of signals. It proposes a decision. Only the Gate applies the authority transition. Each invocation creates an immutable `GateEvent` that records policy, inputs, decision, relevant before-and-after state, contradiction state, reason, and authority version.

The reference implementation ships two current policies and one compatibility policy. The `exploratory` policy permits qualifying recurrence or verified external support to propose a positive change when no unresolved contradiction blocks the seed. It remains the manager-level and evaluation default, and recurrence leaves the external evidence count unchanged. The live conversation default is `evidence_backed`, which requires verified external support. Live callers submit such support through an explicit API that rejects non-evidence kinds, opposing or unverified input, and missing source references before invoking the Gate. `legacy_evidence_required` preserves historical thresholds for compatibility callers. Policy names matter because a Gate event must be interpretable in terms of the rule that produced it.

A useful abstraction is

$$G(P, A_t, \Sigma_t) \to (A_{t+1}, g_t), \tag{5}$$

where P is a named policy, A_t is the authority snapshot, Σ_t is the offered signal set, and g_t is the resulting Gate event. Policies can propose. Seed mutation remains exclusive to the Gate.

4.5 Authority versioning

Authority-changing transitions increment v_t. The version changes when weight, accepted evidence count, or contradiction score changes, and when the seed crosses the promoted boundary. Pure observation changes, such as trace decay without an authority effect, need not advance the version.

Versioning prevents a past authorization from being reused after the state changes. A point-of-use decision must refer to a Gate event associated with the seed's current authority version. If a seed was promoted at version 7 and a contradiction changes its authority state to version 8, every later action requires version-8 authorization.

4.6 Contradictions and recovery

Contradiction is represented by explicit records in addition to the legacy scalar penalty. A record has an identifier, reason, optional source reference, strength, lifecycle state, and, once closed, a resolution basis. An open contradiction blocks influence in the default safety configuration.

Resolution is an auditable action. It closes the blocking record, records a resolution event, and leaves the previous weight unrestored. Authority can rise again only after validation. Resolving a contradiction requires an explicit basis beyond reappearance of the same hypothesis.

This distinction blocks an unsafe shortcut in which a seed is contradicted, recurs several times, and regains authority without any explicit treatment of the contradiction.

4.7 Point-of-use authorization

Promotion is a prerequisite for influence in the reference design. By itself it grants no final permission. The point-of-use contract makes the final decision when a candidate is about to affect an action. Under the recommended configuration, a seed is allowed only when

$$\begin{aligned} \mathrm{allow}(s_t, a) = 1 \quad \text{only if} \quad & z_t = \mathrm{PROMOTED}, \\ & w_t > 0, \\ & \nexists c \in C_t : \mathrm{open}(c), \\ & \exists g : \mathrm{authorizes}(g, s_t, v_t). \end{aligned} \tag{6}$$

The action a may be retrieval, answer modification, a probe, a warning, or another downstream use. The implementation can relax the contradiction check for specialized use. The equation above therefore describes the default safety profile; arbitrary in-process Python is outside that guarantee.

The decision and its record are created atomically by `decide_and_record`. Allowed and denied attempts both enter the audit trail. The authorizing Gate event is selected strictly: it must belong to the same seed, match the current authority version, and represent a non-blocking state that left the seed promoted. A stale event is denied. The runtime never falls back to the last known promotion.

4.8 Live and evaluation conversation modes

The live session selects eligible, previously authorized seeds and applies the point-of-use contract before one model generation. The visible answer is the answer stored in conversation history. Detection runs on that visible output. When any seed was supplied to generation, every candidate detected

in that same answer is deferred rather than ingested or credited as recurrence. This fail-closed rule is broader than embedding-based attribution: semantic distance cannot prove that a differently worded consequence was independent of SSL input. It can defer legitimate new candidates, which is the deliberate cost of a hard no-same-turn-self-credit contract.

The evaluation session retains the historical isolated comparison arm. It first generates and stores an uncontaminated baseline, and it may then generate a separate SSL-assisted answer for measurement. Baseline isolation is therefore an evaluation property, not the product conversation model.

Promotion creates eligibility for surfacing. A shared policy can still exclude the seed using contextual relevance, early-turn constraints, top-k selection, and resurface damping. Damping is applied only after a seed was actually authorized and supplied to the model.

4.9 Prompt-data boundary

A surfaced seed remains candidate data. The implementation encloses surfaced candidates inside explicit delimiters marked as untrusted and instructs the model to treat imperative-looking text inside the block as quoted content. It also bounds the number of seeds and the number of characters per seed and per block [4].

The prompt-data boundary has a narrow function: structural separation of untrusted candidate data. It offers no prompt-injection guarantee. AgentDojo and related work give ample reason to avoid such a claim [9]. A capable model can still respond incorrectly to instruction-like data. SSL places the stronger control before the prompt: only an authorized seed may reach the surfacing stage. The quoting boundary reduces ambiguity at presentation time and leaves audit markers for instruction-like patterns.

5 Reference implementation

The research artifact is implemented in Python. Shadowseed Pro 0.4.2 is the published software release cited here; the reviewed implementation revision is identified separately in Section 6 because the live-runtime architecture continued to evolve after that release [4]. The runtime is split into modules for intake, lifecycle, contradiction records, validation, surfacing, trusted-source handling, retrieval, and point-of-use authorization. A manager object coordinates state and compatibility APIs. New Gate-controlled authority decisions converge on one executable Gate engine.

Authority-bearing fields are guarded by the normal runtime API. Construction rejects arbitrary weight, and direct assignment to authority fields is rejected. Runtime transitions that change authority use one manager transition path. Restoration is treated separately because loading persistent state must reconstruct earlier authority without pretending that a new validation decision occurred. Persisted snapshots are validated before installation, restore their existing authority version, create no new evidence, and create no Gate event. The package includes explicitly named unsafe hooks for tests and edge-case construction. Those hooks are outside the supported runtime contract. Accordingly, the paper limits the non-bypassable claim to supported public APIs and makes no mathematical claim about arbitrary Python code.

The trusted-source layer stores verified material separately from uncertain seeds. Model-generated proposals enter as unverified observations. Retrieval, human feedback, or trusted-source matches can become external validation signals only under their stated provenance rules. This keeps a candidate from serving as evidence for itself.

A local Workbench provides a practical inspection surface for testers. It supports persistent sessions, seed and ledger inspection, scenario runs, baseline/SSL comparison, audit-only feedback,

and verified report or support-bundle exports. The Workbench is an application layer over the runtime. Tester feedback remains audit-only unless it is separately admitted as evidence through the normal Gate path.

6 Assurance evaluation

6.1 Evaluation question

The present evaluation asks a narrow question: *does the reference implementation enforce the authority contracts described in Section 4?* Detector recall, answer benefit, and user preference are deferred to separate empirical studies.

The reviewed live-runtime implementation is commit

`af2184d8c5a7cb1b3560e3cc554856613305a501`.

The repository path executes linting, the test suite on Python 3.10 and 3.12, package build, clean wheel installation, command-line smoke tests, Workbench checks, cross-platform portability checks, and release-asset verification [4].

6.2 Contract coverage

Table 1 summarizes representative contracts and the test families that pin them. The table reports what each test family establishes. Test count is not used as an empirical quality metric.

6.3 Continuous-integration result

The repository records the exact test count and branch coverage for each reviewed commit rather than treating numbers from an earlier release as properties of later code. The configured branch-coverage floor is 80%. CI also requires Ruff, Python 3.10 and 3.12 tests, package build, and clean installed-package command-line smoke tests. These checks are commit-specific evidence and must be rerun on the revision intended for merge [4].

These numbers show that the repository's executable contracts ran successfully under the recorded environment. They measure contract execution in that environment. Statistical confidence about omission quality, usefulness, security under adaptive attack, and human outcomes requires independent evaluation.

6.4 What this evaluation establishes

The assurance evidence supports six implementation-level claims. Uncertain candidates have a representable state in which they persist without steering. New Gate-controlled authority decisions in the supported runtime pass through the documented validation engine. Provenance and contradiction state participate directly in authority accounting. Actual influence requires a fresh point-of-use decision tied to the current authority version. The live session performs one generation and remembers the answer shown to the user. The evaluation session keeps baseline and SSL-assisted paths separately observable.

Semantic atomicity, factual correctness, completeness, usefulness, and Gate-policy optimality remain unevaluated. Contract tests cannot answer those empirical and normative questions.

Contract	Failure prevented	Representative test evidence
New seeds begin weightless	Detector output gains authority at creation	`test_authority_encapsulation.py`; `test_models_extraction.py`
Trace and authority remain separate	Recurrence or reactivation grants steering implicitly	`test_lifecycle_ttl.py`; `test_lifecycle_extraction.py`
Gate-controlled decisions use one engine	A compatibility or helper path creates a second authority route	`test_gate_path_unification.py`; `test_gate_boundary_completion.py`
Recurrence is distinct from external evidence	Repetition masquerades as independent support	`test_gate_signal_routing.py`
Open contradictions block use	A contradicted seed remains silently influential	`test_contradiction_lifecycle.py`; contradiction extraction tests
Point-of-use requires current authorization	A stale promotion authorizes a later action	`test_point_of_use.py`; `test_agent_safety_contract.py`
Live uses one generation and stores its visible answer	Hidden baseline history diverges from the conversation the user read	`test_live_runtime.py`
Same-turn live detection fails closed after SSL influence	SSL-generated text earns recurrence credit for itself	`test_live_runtime.py`
Evaluation history stays baseline-isolated	The controlled comparison arm feeds on its own intervention	`test_shadow_chat.py`; `test_live_runtime.py`
Live evidence enters through a provenance-checked API	The evidence-backed default has no usable or auditable support route	`test_live_runtime.py`
Strict replay checks allowed influence records	An incomplete or mismatched audit record is accepted as valid	point-of-use replay tests

Table 1: Representative executable contracts in the reviewed reference implementation. Test names identify implementation evidence; independent empirical validation requires separate studies.

7 Security and failure analysis

7.1 Detector error

A detector can invent an omission that is already covered, irrelevant, outside scope, or nonsensical. Atomicity heuristics can split a coherent issue badly or fail to split a compound candidate. Weightless creation limits immediate damage. A weak detector can still fill the store with noise and consume reviewer attention. Rate limits, quality calibration, and stronger spam controls remain open work.

7.2 Correlated recurrence

Repeated detection may reflect a real unresolved issue. It may equally reflect a stable detector bias. The exploratory policy deliberately allows recurrence to contribute to promotion under its stated rules, so it suits research settings where investigation is cheap and false promotion is visible. The evidence-backed policy is the more defensible choice where influence should depend on external support. Neither policy can infer source independence from repeated model output.

7.3 Evidence poisoning

Provenance makes evidence inspectable while source trust remains a separate problem. The live evidence API validates signal type, support direction, a caller-supplied verification marker, and source identity; it does not authenticate the source or establish truth. The operator or host application is therefore the trust anchor and must verify a source before setting the marker. Hard-coding it would collapse the intended distinction between observed recurrence and external support. PoisonedRAG shows that retrieval stores can be attacked directly [8]. A compromised trusted source can therefore produce well-formed, provenance-carrying evidence that is still false. Deployment in consequential domains would need source authentication, trust policies, conflicting-source handling, and review outside the current artifact.

7.4 Instruction-like seed text

A seed can contain text that resembles a system instruction or prompt-injection payload. The prompt boundary quotes and bounds that text. The point-of-use contract controls whether the seed reaches the prompt. The model can still mishandle quoted adversarial content. The design reduces one authority ambiguity. Prompt injection remains possible. AgentDojo illustrates why external data and instructions cannot be assumed to remain cleanly separated by model behavior alone [9].

7.5 Persistence boundary

Restoration bypasses new Gate decisions by design because it reconstructs prior state. The implementation validates snapshots and preserves their authority version. The persistence store remains a trust boundary. Production use would require stronger access control, schema migration guarantees, rollback, and durable audit integrity. The current immutable Gate and influence records are in-process data structures. Durable append-only and cryptographically chained audit guarantees are absent.

7.6 In-process escape hatches

Python permits callers with code execution to reach internals. The repository contains explicitly unsafe test hooks. The system's "non-bypassable" property is therefore scoped to supported public runtime paths for new authority decisions. The scope excludes sandboxing and arbitrary in-process code.

8 Discussion

The practical value of separating memory from authority is easiest to see in ambiguous cases. A system may notice the same candidate omission on three turns. Deleting it would discard a pattern worth investigating. Treating it as fact would overstate what recurrence can show. SSL allows a third state: remember the pattern, keep its epistemic status visible, and wait for a policy-relevant reason before permitting influence.

The same distinction clarifies retrieval. A seed can be authorized to initiate a search without treating the search result as automatically true. Retrieved support retains its source identity and enters validation as a typed signal. If contradictory material appears later, the authority version changes and earlier point-of-use authorization becomes stale. This creates a chain that can be reconstructed from candidate to evidence to Gate event to action.

The architecture also makes policy disagreement explicit. A research prototype may accept recurrence as enough reason to surface a candidate for exploration. A higher-risk system may require verified external support. Putting that difference into named policies is preferable to hiding it in a threshold whose semantics are unclear. Domain owners still choose the appropriate policy. The architecture constrains where that policy acts.

There is a cost. More state means more bookkeeping, more failure modes, and more material to review. A Validation Gate that nobody inspects becomes ceremony. Provenance that identifies a source without measuring its credibility can give a false sense of rigor. Audit logs can also expose sensitive text. The architecture earns its complexity only where preserving uncertain context is useful enough to justify an explicit authority trail.

9 Limitations and empirical work still required

The strongest current limitation is empirical. General answer-quality improvement remains unestablished. Deterministic fixtures demonstrate wiring and state transitions. Contract tests establish executable invariants. Neither tells us how often real detectors propose useful absences, how often valid seeds improve a response, or how often surfacing distracts the model.

A suitable next study should freeze the implementation and preregister the evaluation. At minimum, it should compare isolated baseline responses with SSL-assisted responses on tasks containing recoverable contextual omissions; include negative controls in which no meaningful omission is present; measure false influence as well as useful influence; and use blinded, independent assessment. The design should report effect sizes and uncertainty; win rates alone are insufficient. Ablations should separate candidate persistence, Gate validation, retrieval, and point-of-use checks. Such a study can test whether the architecture's control costs buy useful behavior.

Generalization is also open. Evidence for seed quality across domains, languages, model families, and long deployments is currently absent. Weight represents steering authority and carries no calibrated probability of truth. The current implementation provides local persistence and a tester Workbench. It lacks the append-only audit storage, mature access control, privacy and retention policy, operational monitoring, rollback procedures, rate limiting, and independent security review expected of a production system.

Security claims remain bounded. A malicious trusted source can still supply false evidence. Prompt-data quoting leaves residual instruction-following risk. The default contradiction block can be relaxed programmatically. Arbitrary Python code with access to unsupported internals is outside the public-API authority guarantee.

These limits define the method's intended use. The current Shadowseed reference implementation is a research artifact for mechanism inspection, controlled experiments, and tester studies. High-impact deployment in healthcare, finance, employment, law, public administration, education decisions, or autonomous action falls outside that intended use.

10 Reproducibility and artifact statement

The paper cites the published Shadowseed Pro 0.4.2 artifact and describes the later reviewed implementation at commit `af2184d8c5a7cb1b3560e3cc554856613305a501`[4]. The repository records the architecture, contract tests, benchmark implementations, release metadata, and CI workflows used for the assurance claims in this paper. Its validation path verifies Python 3.10 and 3.12, package build, clean wheel installation, Workbench checks, cross-platform portability, and release assets.

A reproduction should record the repository commit, Python and operating-system versions, dependency versions, backend and model identifiers, immutable model revisions where available, input hashes, prompt variants, random seeds, and output hashes. Real-model labels or reviews should be preserved as artifacts because hosted model outputs can vary across runs.

The repository is publicly visible for inspection, research discussion, and evaluation. At the time of writing the repository carries no open-source license; the original content remains all rights reserved. Public visibility grants inspection access only under the repository's stated rights position.

11 Conclusion

A language-model system can benefit from remembering a suspicion before it is ready to trust it. That requires more than a memory store. The representation must preserve uncertainty after storage, validation must have an explicit authority boundary, and permission must still be current when the memory is used.

Shadow Seed Learning implements that separation with weightless candidate omissions, a distinct trace signal, typed validation, provenance-bound evidence, explicit contradiction records, versioned Gate events, and atomic point-of-use decisions. The reviewed reference implementation provides executable evidence that these contracts are wired into the supported runtime. General answer improvement remains an open empirical question for controlled evaluation. Further mechanism expansion cannot substitute for that study.

References

[1] Harvey Yiyun Fu et al. "Absence Bench: Language Models Can't See What's Missing". In: *Advances in Neural Information Processing Systems*. Vol. 38. Datasets and Benchmarks Track. 2025. URL: https://proceedings.neurips.cc/paper_files/paper/2025/hash/36b31e1bb8ecd4f4081686448e9eff2d-Abstract-Datasets_and_Benchmarks_Track.html.

[2] Belinda Z. Li, Been Kim, and Zi Wang. "QuestBench: Can LLMs Ask the Right Question to Acquire Information in Reasoning Tasks?" In: *arXiv preprint arXiv:2503.22674* (2025). DOI: 10.48550/arXiv.2503.22674.

[3] Di Wu et al. "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory". In: *International Conference on Learning Representations*. 2025. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html.

[4] E-AI-MODEL. *Shadowseed Pro 0.4.2*. Software research artifact. Version 0.4.2, commit `1803 9196549e68936d1976b74eb8e6aac3eac98e`. All rights reserved. 2026. URL: https://github.com/E-AI-MODEL/shadowseed-pro.

[5] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[6] Kelvin Guu et al. "Retrieval Augmented Language Model Pre-Training". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 3929–3938. URL: https://proceedings.mlr.press/v119/guu20a.html.

[7] Noah Shinn et al. "Reflexion: Language Agents with Verbal Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. DOI: 10.52202/075280-0377.

[8] Wei Zou et al. "PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models". In: *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 3827–3844. ISBN: 978-1-939133-52-6. URL: https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag.

[9] Edoardo Debenedetti et al. "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents". In: *Advances in Neural Information Processing Systems*. Vol. 37. Datasets and Benchmarks Track. 2024. DOI: 10.52202/079017-2636.

[10] Soyeong Jeong et al. "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity". In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 7036–7050. DOI: 10.18653/v1/2024.naacl-long.389.

[11] Akari Asai et al. "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In: *The Twelfth International Conference on Learning Representations*. 2024. URL: https://openreview.net/forum?id=hSyW5go0v8.

[12] Charles Packer et al. "MemGPT: Towards LLMs as Operating Systems". In: *arXiv preprint arXiv:2310.08560* (2023). DOI: 10.48550/arXiv.2310.08560.

[13] Shehzaad Dhuliawala et al. "Chain-of-Verification Reduces Hallucination in Large Language Models". In: *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 3563–3578. DOI: 10.18653/v1/2024.findings-acl.212.

[14] Sebastian Farquhar et al. "Detecting Hallucinations in Large Language Models Using Semantic Entropy". In: *Nature* 630 (2024), pp. 625–630. DOI: 10.1038/s41586-024-07421-0.